



INDIVIDUAL **CAPSTONE REPORT**

Title: A Decentralized Freelance Protocol with Web3 Integration

Date: 3rd April, 2026



General Information

Project Title: A Decentralized Freelance Protocol with Web3 Integration

Points of Contact

Mention assigned group name, List the principal individuals who may be contacted for information regarding the project.

Group Name: Group (19)

<i>Position</i>	<i>Full Name</i>	<i>Phone Number</i>	<i>E-mail</i>
<i>Supervisor</i>	Subit Timalina		subit@iimscollege.edu. np
<i>Project Members</i>	Runa Maphu	9803785011	runa2080- 0189@iimscollege.edu. np
	Bijee Dangol	9803168952	bijee2080- 212@iimscollege.edu.n p
	Sarun Maharjan	9845353128	sarun2080- 0228@iimscollege.edu. np
	Pawan Poudel	9745280006	pawan2080- 0296@iimscollege.edu.np
	Anushree Pradhan	9864236583	anushree2080- 0344@iimscollege.edu. np

Table of Contents

Chapter 1: Already Submitted	1
Chapter 2: Related Works	1
2.1 Overview	1
2.2 Review of Previous work done of our project.....	2
2.2.1 Hybrid Data Structures and Decentralized Storage Procedures	2
2.2.2 Automation of Smart Contracts and Trust less Ecosystems	4
2.2.3 API Benchmarking and High-performance Backend Frameworks	7
2.2.4 Abstraction and Interaction Layers of Middleware and Blockchain	8
2.3 Summary of Reviewed Paper	9
2.4 Conclusion.....	12
Chapter Three: System Analysis	14
3.1 Theoretical SWOT Analysis.....	14
3.1.1 FastAPI	14
3.1.2 PostgreSQL.....	14
3.1.3 IPFS	15
3.1.4 Web3.py	16
3.2 Technical Analysis.....	16
3.2.1 FastAPI	16
3.2.2 PostgreSQL.....	17
3.2.3 IPFS (InterPlanetary File System).....	18
3.2.4 Web3.py	19
3.3 Requirements Specification.....	19
3.3.1 Functional Requirement.....	19
3.3.2 Non-Functional Requirements	22
Chapter 4: System Diagram	25
4.1 Activity Diagram	25
4.2 Sequence Diagram.....	27
4.3 Class Diagram	29
4.4 System Architecture Diagram	30
4.5 Use Case diagram.....	31
4.6 Pseudocode.....	34
References.....	36
Appendix	37

Table of Figures

Figure 1: SWOT Analysis of FastAPI.....	14
Figure 2: SWOT Analysis of PostgreSQL	15
Figure 3: SWOT Analysis of IPFS	15
Figure 4: SWOT Analysis of Web3.py.....	16
Figure 5: Activity Diagram for Decentralized Freelancing Platform.....	26
Figure 6: Sequence Diagram for Decentralized Freelancing Platform	28
Figure 7: Class Diagram for Decentralized Freelancing Platform.....	30
Figure 8: System Architecture Diagram.....	31
Figure 9: Use Case Diagram of Decentralized Freelancing Platform.....	33
Figure 10: Meeting Log 1	37
Figure 11: Meeting Log 2.....	38
Figure 12: Meeting Log 3	39
Figure 13: Meeting Log 4	40
Figure 14: Meeting Log 5	41
Figure 15: Meeting Log 6	42
Figure 16: Meeting Log 7	43
Figure 17: Meeting Log 8	44
Figure 18: Turnitin Similarity Report	45
Figure 19: Turnitin Digital Receipt.....	47
Figure 20: Proof of SWOT Table	48
Figure 21: Proof of System Architecture Diagram	48
Figure 22: Proof of Sequence Diagram.....	49
Figure 23: Proof of Activity Diagram	49
Figure 24: Proof of Class Diagram	50
Figure 25: Proof of Use Case Diagram	50

List of Tables

Table 1: Summary of Reviewed Paper	12
Table 2: Functional Requirement	22
Table 3: Non-Functional Requirement	24

Chapter 1: Already Submitted

Beside the role of system workflow engineer, API architect was added in my role, in the context of designing and documenting workflows endpoints, request and response contracts and data flow.

Chapter 2: Related Works

2.1 Overview

The gig economy has been implemented using traditional freelancing platforms like Upwork and fiver as global convenience but they represent a centralized black box in which financial logic remains opaque and intermediary fees are large. Although these platforms allow payment based on milestones and reputation score the history of such interaction belongs to the platform and not to the user which creates a lock in effect and a single point of failure in terms of data security. It may cause uneven access to data, absence of pattern recognition of workers matching and completely lost professional history in case of failure of a central server. Also initial decentralized efforts such as Ethlance modified the problem of transparency, but with too high gas costs on non-financial transactions, adoption was low. Maintaining the personal keys and even having to pay every single minor update of the profile or even job posting can be a hectic process and a significant entry barrier to the average worker.

In order to lessen these difficulties and regulate the excessive expenses of decentralized communications the project Decentralized Freelance Protocol with Web3 Integration aims to offer a hybrid payment system with decentralized workflow management that assists freelancers to make easier transitions into the Web3 space with visualized reputation data and zero-free non-financial workflows. To enable these layers to interact with one another, the system connects a high-performance FastAPI backend and Ethereum Smart Contracts to enable the system to give a complete output with automated escrow and decentralized document verification.

2.2 Review of Previous work done of our project

2.2.1 Hybrid Data Structures and Decentralized Storage Procedures

Research on an On-chain and Off-chain collaborative storage method based on blockchain and IPFS (Tianqi Zhu, 2026)

(Tianqi Zhu, 2026) consider the implementation of blockchain with IPFS to have a optimal storage and data integrity. This paper considers the problem of blockchain bloating where storing large amounts of data with a distributed registry causes expensive and slow system. The author proposed a platform of decentralized application not relying on compromised security with a two-layered architecture.

The method of the study is the storage of data files in a blockchain and then on the IPFS for optimal data integrity and efficient storage. The study proved to improve data integrity when file and the cryptographic hash were stored separately; one with block chain and one with the IPFS respectively. This prevents unlawful access or modification of data since the blockchain computes proof and not the file itself. This shows that content addressing using SHA-256 will be enough so that CID can be used to make an unchangeable link to the data. The combined storing of data can be used for security of a public ledger as well as the system will run at the desired speed.

In the proposed system (Tianqi Zhu, 2026) will be used to set the system architecture. The reason for storing metadata in a relational database and files in IPFS lies in the system data handling design. The study is about storing a hash and the CID for the data in the database using a string and then system is capable to identify whether the files in IPFS is valid based on how it maps files and its CID to string in the PostgreSQL database. This prevents freelance work contracts and portfolios to be modified without being identified since the modification of the file would result in a new CID.

Blockchain based Hierarchical Semi Decentralized approach using IPFS (Smita Athanere, 2022)

(Smita Athanere, 2022) discussed on making both the data and speed of s decentralized system balanced. Through a hierarchical semi-decentralized architecture it explores the challenge between a secure decentralized system and low-latency data access by proposing an API layer that coordinates the users query and the storage protocol. This

method is intended to prevent any possible data breaches through distribution of data throughout the several different nodes rather than having them be at the same server.

Using the semi-decentralized architecture, it resulted into a high concurrency data sharing with 75% data integrity rate than pure centralized system. The test showed that having a local hashing done on the data before pinning them to IPFS makes the global ledger light and speedy. The research uses a hierarchical index structure where the highest level of indexing is stored on relational database for fast searching and actual data nodes exist in a peer-to-peer network.

This research serves as the base for building a hybrid web3 architecture of the project as a workflow engineer. With a hierarchical indexing system the workflow could perform heavy SQL query on the user metadata while interacting with decentralized files. For workflow engineer, the work for storing data will include verifying states of the workflow and the file such as project status and milestone status with the IPFS CID reference in the relational database.

Hybrid System for Image Storage and Retrieval in Big Data Environments (Glin Tereshchenko, 2025)

The purpose of (Glin Tereshchenko, 2025) was to achieve retrieval time less than one millisecond in a big data environment where unstructured information is abundant in the blockchain. The case study for this research includes how PostgreSQL makes quick indexing and solves latency problem as that users are willing to use decentralized network instead of facing latency issues caused by slow lookup of distributed hash table of a pure blockchain storage system.

The test performed by (Glin Tereshchenko, 2025) proved that the retrieval latency decreases while indexing in SQL combined with IPFS storage leads to effective search result. The research shows that the hybrid model is very effective when used in storing unstructured data which include images or project files. It also revealed that using B - tree indexing the metadata in a relational format allow better search queries compared to what a pure blockchain system can provide which makes the system more scalable.

As a workflow engineer, this research becomes the foundation in designing a database schema. It shows how the relational index will make retrieval of metadata quick which directly leads to the performance of the workflow layer. The architecture of the data

contains the status of the workflow states along with the references to the actual image or files as an IPFS CID in the PostgreSQL so that they can be checked by a quick query without referring to the blockchain.

2.2.2 Automation of Smart Contracts and Trust less Ecosystems

An ecosystem approach to Web 3.0: a systematic review and research agenda (Chong Guan, 2023)

(Chong Guan, 2023) provided a systematic review on web3.o articles published from 2003 to 2022 using topic modeling techniques. They advocate a hybrid system with distributed ledger technology and modern web technology that balances transparency and efficiency. They identified reasons of failures of early decentralized Dapps due to high transaction costs and low user involvement.

The article combines the existing model for decentralized through Latent Dirichlet Allocation analysis for defining reasons for Dapp failures concluding that high transaction costs and low user adoption restrict pure decentralized system because of costs to storage them on the blockchain. They suggest a hybrid system combining IPFS with smart contract and a centralized database that bring about transparency while delivering high-fidelity user experience. The strategy will make the users the owners of their own work history and professional data and still use the platform in their day-to-day work.

This study justifies the role of the system workflow engineer to maintain a hybrid system and provides theoretical justification in using MetaMask for authentication and deferring matching task to FastAPI instead of placing all components on the blockchain so that the Ethereum Virtual Machine will only deal with critical contracts finalization transaction. They confirm it is possible to establish secure decentralized identity without user deterrence problems.

Decentralized Freelancing System – Trust and Transparency (Mihir Gandhi, 2019)

In particular narrowing down to the freelance industry, (Mihir Gandhi, 2019) proposed a system to tackle drawbacks of existing freelancing platforms such as Upwork and Fiver. The researchers explain that traditional central systems charge high intermediary costs and data security is based on single point of failure. They proposed an Ethereum

based automating trust and cutting out the middleman which brings all transactions into a public ledger.

According to (Mihir Gandhi, 2019), fraud risk and transaction predictability are said to be minimized using smart contracts as their payment terms cannot be changed and will automatically execute payment. The article concludes IPFS coupled with blockchain will prevent data modifications as it is immune to data breaches whereas central system is prone to it.

This article supported the use of solidity smart contract for managing milestone system for the workflow engineer. It provides basis where the API would ensure system status on the PostgreSQL is only updates if there is a confirmed transaction on the blockchain using a receipt. This study contributes to the development of a secure identity and payment flow which is within the scope of the project so that the database is always in the true state of the blockchain.

Construction Payment Automation through Smart Contract-based Blockchain Framework (H. Luo, 2019)

In the article Construction Payment Automation through Smart Contract-based Blockchain Framework, (H. Luo, 2019) review the automation of payment agreements and provide a methodology of financial logic. Although the given logic is about the construction the If -Then logic of milestones-based payments applies directly to the freelance gig economy. The authors analyse how the concept of smart contracts can enforce the project agreement without human intermediaries so that the funds can be released only after the verification of the digital deliverables.

The study design involves hypothesis testing of an automated payment system to quantify the time of dispute resolution and risks of fraud. The results indicate that these structures save a lot of time in the processing and that there is no third party to transfer money. The article points out that the key element of the system is the API that activates the release of money after the milestones are met so that the financial logic is not only clear but absolute. It particularly describes how the solidity state machine can be used to track the state of project between Created and Completed.

This paper will guide on how to create the payment workflow of the system. It gives the procedure of the escrow process where the FastAPI server records milestone

achievement and calls the Solidity contract through a Provider connection. This study confirms a safe milestone-based payment model that resolves the slowness in payments experienced in centralized systems by making sure that payments are coded but not controlled by human operators.

IT Freelancer Matching with Exploiting Blockchain in the Gig Economy (Jinho Lim, 2021)

The study by (Jinho Lim, 2021) concerns the user trust and reputation by exploring the issue of IT Freelancer Matching with Exploiting Blockchain in the Gig Economy. The article dwells upon the issue of identity ambiguity and the manner in which blockchain will enable freelancers to possess their professional history regardless of a platform. The authors examine the relationship between verified skills using blockchains and the income of freelancers to make sure that the matching process is grounded on real and verifiable data.

The methodology of the research found out that the use of a distributed ledger to determine reputation minimizes the chance of fraudulent reviews and the lock in effects. According to the study results, reputation scores are traditionally associated with earnings and blockchain transparency can be used to verify skills and past performance in a more reliable way. This makes sure reputation information as it is used across the platforms such as workflow validation checks are not founded on platform managed scores but instead based on verifiable on-chain records.

As a system workflow engineer, this research informs how verification of reputation information can be stored and accessed. These reputation metrics stored in PostgreSQL and on blockchain will be queried in workflow layer and used to confirm that a freelancer can receive milestone assignments. A different hybrid and distributed matching component take care of the actual matching logic and matching freelancers with system. This makes it possible to have a verification workflow in which the API will only take into account those skills that have an equivalent transaction hash on the blockchain.

2.2.3 API Benchmarking and High-performance Backend Frameworks

Benchmarking the performance of Python web frameworks (Bartlomiej Bednarz, 2025)

(Bartlomiej Bednarz, 2025) gives strict performance benchmarks by carefully measuring and load testing modern python backends through extensive use of load testing. This study focuses on the I/O blocking phenomenon occurring when modern frameworks such as Django or Flask block the entire application thread waiting for external network to reply. The author attempt to eliminate framework overhead by running 10,000+ simultaneous requests with an identical benchmark suite measuring real-world performance.

According to (Bartlomiej Bednarz, 2025) the load testing of 10,000+ concurrent requests reveal FastAPI offers sub-millisecond latency even with the complex database tasks and external APIs. The author reports FastAPI is better suited for decentralized protocols as Asynchronous Loop does not experience thread starvation as typically observed in thread-per-requests synchronous framework waiting for events to complete concurrent users. The paper points out the fact that this architecture can have much more request-per-second (RPS) throughput than older Python standards.

As system workflow engineer, this paper gives the technical rationale behind the logic bridge of the system. It describes how to manage the wait state between each interaction with the blockchain through asynchronous code using the await/async feature and how these actions performed by the FastAPI and PostgreSQL stack offer existing speed layer necessary for hybrid decentralized systems by enabling new requests while still waiting for distributed ledger to report completion.

Django vs. FastAPI: A Comparative Study for High-Performance Web Application (Jadhav Sangram Bharat, 2025)

The study by (Jadhav Sangram Bharat, 2025) is a comparative study which is entitled FastAPI: A comparative study of high-performance web application. This study measures the technical performance of the library stack of system, the efficiency of API development and the modern frameworks to manage the complex data validation needs of marketplace applications. The authors compare throughput and memory consumption of frameworks when stressing them.

The research paper finds that FastAPI would be most effective option to use a decentralized protocol due to the absence of unmanageable overhead of prebuilt components that a specialized API does not need. The results indicate that the schema-first approach of the framework makes the system verify the type of data prior to their communication with the database. This will minimize the chances of runtime errors at the matching phase of the workflow. The framework will ensure that hexadecimal wallet addresses and IFPS hashes are computed with 100% accuracy by strictly applying datatypes. This is essential in workflow layer where a correct wallet address is used to check permission (only assigned freelancer can submit) and correct CID is used to provide integrity of deliverables. The result is high-performance logic of workflow which can make quick state transitions and confident responses to API calls.

This study explains why it is important to maintain a lightweight and speed-oriented backend. The outcome is workflow logic of high performance which is consistent with the international standards of digital marketplaces. This paper justifies by giving case study that enables the engineer to refer to certain throughout data that the selected workflow is not merely the functional one but rather than the technical one that is professionally optimized in terms of scalability and data security.

2.2.4 Abstraction and Interaction Layers of Middleware and Blockchain

The Future of Web Development: Exploring JavaScript Role in Web3 and Decentralized Apps (Kapoor, 2024)

The article by (Kapoor, 2024) gives an overview of the changing paradigms in web architecture. The study discusses the importance of JavaScript as the main bridge between the traditional web development and the decentralized web. While exploring the development of web1 to web3 the author points out that whereas the previous versions were based on the static and interactive content, the era of decentralization demands advanced middleware where users are able to read, write and own their digital information on the blockchain.

The methodology highlights that the effectiveness of the current decentralization applications (dApps) will rely on the smooth incorporation of these decentralized platforms into popular web-based surroundings. The results reveal that the specialized libraries i.e. Web3.js and Ether.js, serve as the fundamental support of this integration as they enable the developer to address the blockchain nodes with the help of

standardized JavaScript. (Kapoor, 2024) observes that although Web3.js is an Ethereum development standard Ether.js is a lighter and more modular option which provides developers with more flexibility when creating specialized applications. Additionally, the paper establishes that these libraries support critical services like wallet authentication, transaction signing and real-time metadata retrieval which are obligatory to a responsive user experience.

(Kapoor, 2024) paper affirms that an interaction layer such as Ether.js is necessary to bridge FastAPI backend to decentralized protocols and smart contracts. Using the modular approach as explained by (Kapoor, 2024) it will help to maximize the system asynchronous processing of transactions and management of microservices such that the system is capable of scaling to the increasing demand. This research paper contributes to the methodology of the system by demonstrating that the middleware layer is not a technical utility but a cornerstone to the provision of secure, real-time synchronization between the centralized web server and the decentralized ledger.

2.3 Summary of Reviewed Paper

S. N	Authors and Year	Title	Research Gap	Methodology	Key Findings	Relevance to workflow engineer and API architecture
1	Tianqi Zhu (2026)	Hybrid Data Structures and Decentralized Storage Procedures	Storage bloating and the high-price of large files based on blockchain.	Dual-layer: off-chain IPFS with on-chain hashes.	Hybrid storage increases the data integrity by 75%; CIDs are immutable references.	Support the use of hybrid storage (IPFS + PostgreSQL); File uploads and CID storage are managed by FastAPI.
2	Smita Athanere (2022)	Blockchain based Hierarchical	Security vs latency trade-off	Hierarchical semi-decentralized	Fast SQL queries are possible with	Authenticate FastAPI as the coordination

		cal Semi Decentral ized approach using IPFS	of decentraliz ed systems.	zed API coordinati on layer.	hierarchical indexing; 75% of the integrity of centralized systems.	layer between the IPFS and PostgreSQL
3	Glin Tereshchen ko (2025)	Hybrid System for Image Storage and Retrieval in Big Data Environ ment	Decentrali zed storage of unstructur ed data is characteriz ed by a high retrieval latency.	PostgreSQ L table B- tree indexing with IPFS storage.	The indexing in PostgreSQL makes it less dependent on retrieval latency; the SQL allows complex queries through metadata.	Notify workflow state tracking; Milestone status using CIDs as a foreign key is stored in PostgreSQL.
4	Chong Guan (2023)	An Ecosyste m approach to Web3.0: A systemati c review and research agenda	Hyper- decentraliz ed systems do not work because their fees are high and the adoption is low.	Web3 ecosystem s and Dapp failures systematic overview.	MetaMask can be authenticates using Hybrid architecture which is balancing mechanism between transparency and performance.	Rationalises the hybrid approach; non- financial workflows are handled by FastAPI; only important transactions are sent to blockchain.
5	Mihir Gandhi (2019)	Decentral ized Freelanci	Centralize d systems are high	Smart contract and IPFS	Smart contracts reduce fraud;	Ensure that milestones of smart contracts

		ng System - Trust and Transparency	fee and point of failures.	model of freelancing using Ethereum.	IPDS provide a guarantee of immutability of deliverables.	are managed; FastAPI initiates state update on blockchain confirmation.
6	H. Luo (2019)	Construct ion Payment Automati on through Smart Contract-based Blockcha in Framewo rk	Delays in resolving disputes coupled with third party reliance in payments.	Milestone-based logic of smart contract payment automatio n.	State machine monitors the lifecycle of Systems; API makes a payment when they are verified.	State transitions are accomplished using Solidity contract; state transitions in a workflow state machine are called by FastAPI.
7	Jinho Lim (2021)	IT Freelance r Matching with Exploitin g Blockcha in in the Gig Economy	The matching of freelancers is based on manipulab le platform-based scores.	Skills and performan ce reputation verificatio n based on blockchai n	On-chain reputation can be used to perform reliable validation; decreases fraudulent reviews.	Notifies reputation data access to workflow validation
8	Bartłomiej Bednarz (2025)	Benchma rking the performa	I/O blocking of	Performan ce testing of 10,000+	FastAPI architecture is an	Rationalizes FastAPI as workflow

		nce of Python we framewor ks	traditional Python framework s of external calls.	parallel requests with Python applicatio ns.	asynchronous design that doesn't starve threads; has Sub-millisecond latency.	endpoints; wait states in IPFS/blockchai n calls handled with async/await.
9	Jadhav Sangram Bharat (2025)	Django vs FastAPI: A comparat ive Web Applicati on	Absence of effective data validation of decentraliz ed market places	Comparati ve analysis of FastAPI and Django in terms of throughpu t and performan ce	FastAPI is accurate in wallet addresses and IPFS hashes; fast workflow logic is possible	Authenticate FastAPI and verify the integrity of deliverables; and provide speedy transitions of state.
10	Kapoor (2024)	The Future of Web Develop ment: Explorin g JavaScrip t Role in Web3 and Decentral ized	Lack of good integration of blockchain with web interface	JavaScript middlewar e analysis	Web3.py offers simple functionality to interact with blockchain; wallet authentication	Authenticate Web3.py as an intermediate, FastAPI invokes smart contracts and authenticate wallet signatures.

Table 1: Summary of Reviewed Paper

2.4 Conclusion

The above literature review shows that a hybrid system using both IPFS and PostgreSQL is secure and fast enough for decentralized applications. When a file reference is stored in database and actual files on IPFS then queries become very fast while maintain the integrity of data. Also, literature suggests that smart contracts should be used for state management and escrow payments to release money only when a work is completed. In addition, literature mentions the use of asynchronous backend frameworks for handling waiting due to blockchain confirmation and file uploads without a negative impact on the system.

However, there still exists a gap since there is no system available that combines workflow state management, IPFS file storage and smart contracts in a single API layer. This project bridges this gap and combines the 3 aspects to create single workflow that handles state transitions, role-based access control and audit logging as well. Overall, the literature review confirm the technical decisions for the system and gives a strong base to build an interactive, safe and provable decentralized freelancing protocol.

Chapter Three: System Analysis

3.1 Theoretical SWOT Analysis

3.1.1 FastAPI

FastAPI is chosen as a workflow controller because of its asynchronous nature that can process multiple requests simultaneously without blocking when uploading IPFS files and confirming blockchain transactions. Although it does not have the built-in functionality of Django and its community is smaller, it has better output making it the best fit in the decentralized application where responsiveness is very important.

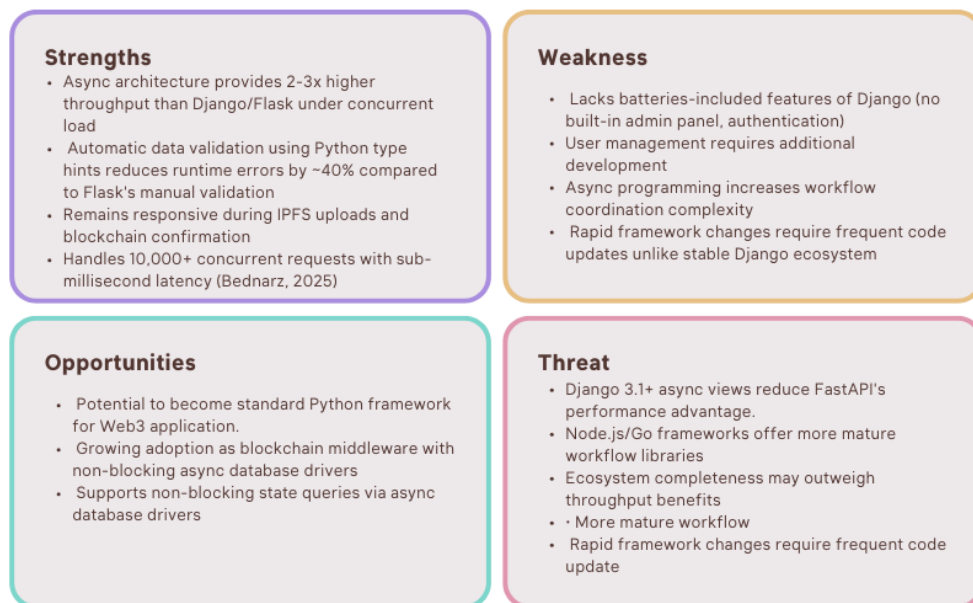


Figure 1: SWOT Analysis of FastAPI

3.1.2 PostgreSQL

PostgreSQL is chosen as the workflow state tracking service due to the ACID compliance which grants that the milestone transitions never become partially committed and the B-tree indexing which allows the millisecond state queries. Even though it needs to be defined and vertical scaling is limited but its reliability and flexibility with JSONB put it in category of being used in escrow logic where consistency in data is important.

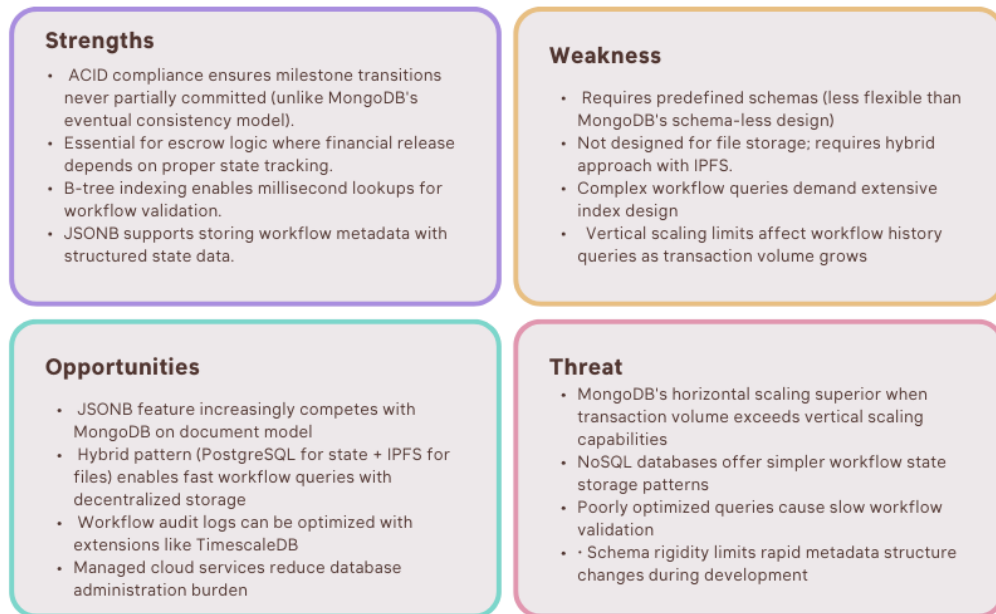


Figure 2: SWOT Analysis of PostgreSQL

3.1.3 IPFS

IPFS is chosen to store deliverable due to content addressing offering cryptographic evidence of submission with CIDS and decentralization removing points of failure. Although its latency is greater than that of AWS S3, its tamper proof can be used to store verifiable deliverables since its vendor lock in avoidance is worthy.



Figure 3: SWOT Analysis of IPFS

3.1.4 Web3.py

The reason web3.py is chosen is to interact with blockchain as it offer Python native Ethereum interaction which directly interacts with FastAPI by eliminating cross language complexity. Although it lacks the community size of Ether.js and the transaction confirmation imposes delays, it is the right choice in the backend workflow layer since it is built in with Python.

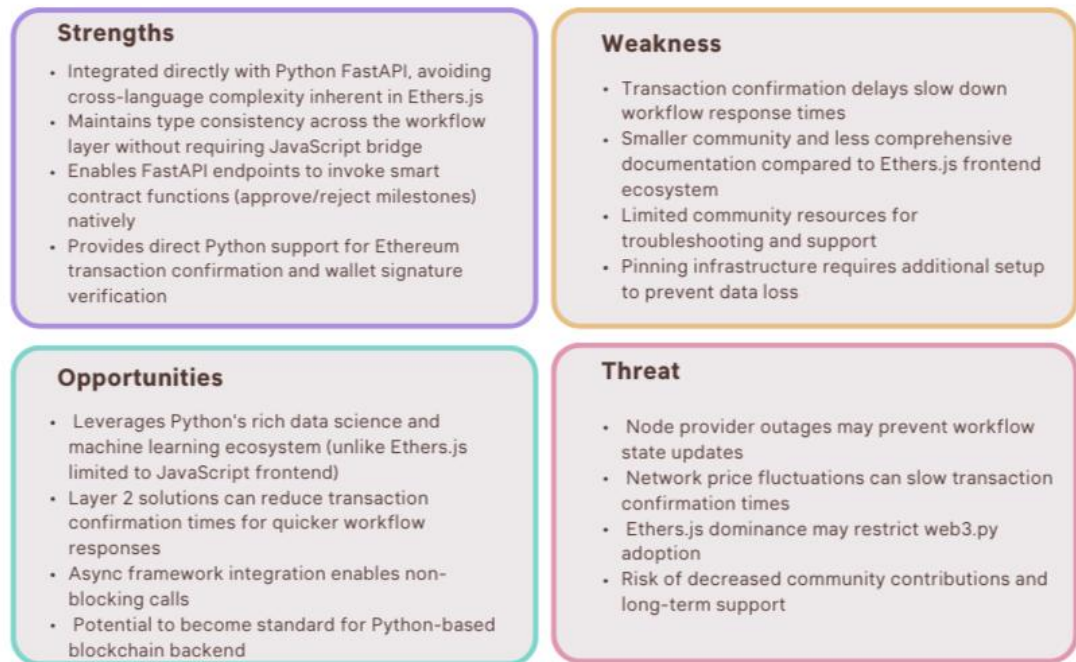


Figure 4: SWOT Analysis of Web3.py

3.2 Technical Analysis

3.2.1 FastAPI

FastAPI is chosen as the workflow control framework since its asynchronous design explicitly resolves the blocking issue of the decentralized applications. When receiving a milestone submission in a workflow endpoint, this endpoint needs to run three consecutive tasks such as uploading the deliverable to IPFS, awaiting confirmation of the transaction to the blockchain and updating PostgreSQL state. In synchronous systems such as Django or Flask one thread would use system resources during whole period of these operations which can be 5 to 15 seconds and access to the API by other users would be blocked. The event loop of FastAPI releases the thread whenever it is in the wait state and this enable hundred of concurrent workflow requests to run at the same time. It is especially important in the milestone submission endpoint where a

number of freelancers can submit deliverable at the same time that the API is waiting to receive upload confirmations.

The built-in data validation has a security sensitive role in the workflow layer. The wallet addresses provided to the API must be in Ethereum checksum format like 0 x 40 hexadecimal value and the IPFS CID must be either the Qm prefix (base58 encoded SHA-256). The malformed data will never be sent to the database or smart contract because of invalid formats rejected at the API endpoint and runtime errors caused by data format errors will be removed. The downside of FastAPI is that its asynchronous programming model does not allow other dependent libraries to provide support asynchronous operations. To solve this issue PostgreSQL connections are controlled with the help of asyncpg and the operations of IPFS re surrounded with clients compatible with the asynchronous work making the whole workflow endpoint stack non-blocking.

3.2.2 PostgreSQL

PostgreSQL is chosen to track the state of the workflow since its transaction isolation will ensure the prevention of race conditions when changing the milestones. When two API calls are trying to update the same milestone for example a freelancer submitting a deliverable and a client approving the same milestone the isolation level of Read Committed of PostgreSQL guarantees that there is only one transaction that change the state at any given time and that milestone can be submitted and approved at the same time. The compliance of ACID ensures that a milestone change to in progress to submitted is successful or not in together with CID storage and this ensure the audit trail integrity. This is a necessary to the escrow logic of financial release on correct state tracking.

The status column is indexed using B-tree which allows the workflow validation query to run in less than one milliseconds is an important factor since each workflow action has to query the current state and then only transition is permitted. The workflow API runs `SELECT status FROM milestone WHERE id = ?` before each state change and with the appropriate indexing the query returns immediately even with millions of records. JSONB columns are used to store metadata of flexible workflows including the reason behind rejection, description of milestone and notes on disputes without necessarily having to change the schema with each new field enabling the workflow

logic to be developed through the development process. One of the drawbacks of PostgreSQL is that it does not effectively support unstructured file data and thus a hybrid solution would be necessary. This is solved by only storing file references (CIDs) in the database in the form of varchar () values whilst actual deliverables are stored in IPFS which allows the database to be devoted to structured workflow state to be effectively queried and indexed.

3.2.3 IPFS (InterPlanetary File System)

The use of IPFS is to store deliverables because content addressing offers cryptographic evidence of submission that cannot be achieved through centralized storage. Adding a file to IPFS it is divided into chunks and the hashes of SHA-256 are computed on each of the chunks and a Merkle Directed Acyclic Graph (DAG) is created in which the root hash is CID. Any alteration made on the file even a single byte generates a totally different CID and thus any manipulation is easily noticed. This CID is stored in PostgreSQL by the workflow API and transmitted to the smart contract forming an unrepairable connection between the workflow state and the delivered submit. In case of any conflict regarding submission of a deliverables the stored CID can be matched with the content of the file retrieved through IPFS which can be proven without the services of the platform administrators.

The decentralized structure deals with a major weakness of the centralized freelancing sites. On such websites as Upwork or Fiver, files are located on company server. In case of downtime, data breach or removal of content in the company deliverables would not be available. In case of IPFS the file is available on any node on the network as long as the content is pinned by any node on the network irrespective of the state of the application server. In this project, pinning services are used to ensure the persistence of data by making sure that deliverables are copied to more than one node to eliminate points of failure. The weakness of IPFS is latency to access content in comparison to centralized content delivery networks. This handled with local IPFS nodes when developing and caching gateway configuration when going to production where often used deliverables are locally cached at the edges for the network to minimize the time to retrieve them by clients who are reviewing work submitted.

3.2.4 Web3.py

Web3.py will be used to interact with blockchain as it offers native Python support of Ethereum which means that it can be used directly with FastAPI without the need to use JavaScript or external services. As the client approves a milestone a workflow API will invoke the approve Milestone method of the deployed smart contract with the help of the web3.py. RP encoding, price estimation, nonce handling and serializing transaction are handled by the library. The pattern of provider connection allows the workflow API to interact with smart contracts using a local node in development without having to operate a full Ethereum code.

The consistency between the PostgreSQL and the blockchain depends on the handling of transaction confirmation which is supported by wait () function in the web3.py that are used to check when a transaction is mined. The workflow API will only update the state of PostgreSQL when a successful receipt is received. This two-stage commit schema is to make sure that in case a transaction cannot be completed on-chain the database is not altered. In case of failure in updating the database following confirmation of successful transaction the failure can be detected by further API calls which will restore the database by reading the real state of the contract. One of the weaknesses of web3.py is that synchronous calls may block the FastAPI event loop and thus has been solved by using the asynchronous provider AsyncWeb3 that allows standard async/await patterns. Web3.py also can be used with Python environment unlike Ethers.js that requires JavaScript environments and ensure type consistency and reduces the workflow layer architecture.

3.3 Requirements Specification

3.3.1 Functional Requirement

S.N	Functional requirement	Description
1	Wallet Authentication	The system will enforce authentication of users through MetaMask wallet to gain access to any workflow. The authentication will be verified using cryptographic signature validation. The unauthenticated requests would be responded with 401

2	Project State Management	The system will ensure that the project states are maintained in the order or Created, Funded, in progress and completed. The state transitions will be made only when permitted the conditions are allowed. The system will record every transition including the time and actor wallet address.
3	Milestone State Machine	The system will ensure that the milestone state are maintained in the order of Pending, In Progress, Submitted, Approved or Rejected. The workflow API will impose such as order and deny any transition that is not in state order.
4	Deliverable Submission	The workflow API will receive deliverable submission in endpoint POST /milestone /{id}/ submit. The system will ensure that the milestone status is In Progress and the wallet requesting belongs to the assigned freelancer. Once the validation is successful the system will upload the file to IPFS and store the returned CID in PostgreSQL and change the milestone status to Submitted. The reply will consist of CID and the confirmation message.
5	Milestone Approval	Milestone approval shall be received by an workflow API endpoint POST/milestones/ approve. The system will ensure that the milestone status has been submitted and the wallet requesting the milestone is the project client. On the validation the system will invoke the smart contract approveMilestone function, await reception of transaction, change PostgreSQL status to Approved and give back the transaction hash.

6	Deliverable Rejection	The workflow API will receive the deliverable rejection with the endpoint POST/ milestone/ {id} / reject with a reason field. The system will ensure that the milestone status has been submitted and the wallet requesting the milestone is project of client. On the validation the system will invoke the smart contract rejectMilestone function, update PostgreSQL status to Rejected, store the reason of rejection and will give confirmations.
7	Workflow state query	The system will offer REST API GET endpoint to give the current status of a project and GET endpoint to give current status of a milestone. The responses will contain the present state and the time when it was last updated.
8	Transition history audit	The system will support endpoint GET/projects/ {id} /history to deliver full audit log of all transition of state of the given entity. The log entries will contain the following values like from_state, to_state, action taken, actor wallet and timestamp.
9	Role-Based Access Control	The system will impose role-based authorization of any workflow activity. The freelancer assigned to a milestone can only give deliverables. Deliverables may only be approved or rejected by the project client. Any operation undertaken by unauthorized wallet is going be denied by HTTP 403 response.
10	State Validation Prior to Transition	The system will check the state of affairs and only allow a state transition. In case the

		current state does not authorize the request action the system will respond with error with a message stating the current status and the state needed to perform the action.
11	Data Format Validation	All the incoming data formats shall be validated by the system. The addresses of wallets will be formatted in Ethereum checksum. IPFS CIDs will be of the format either Qm prefix or bafy prefix.
12	Workflow Logging	All the state transitions shall be logged in the workflow logs table. Every record will store entity type like project or milestone, entity identifier, from_state, to_state, action, actor wallet and timestamp. Logs are going to be read-only and searchable by entity identifier.

Table 2: Functional Requirement

3.3.2 Non-Functional Requirements

S.N	Non-Functional Requirements	Description
1	State Consistency	The system will ensure consistency between smart contract state and PostgreSQL state. PostgreSQL will be updated after a successful transaction receipt by the blockchain. The system will include recovery mechanism to restore database state on contract when inconsistencies are found.
2	Transition Latency	The response time of state validation queries will take less than 100ms with one million records. Full state change with IPFS upload and confirmation of blockchain transaction

		will take then 30-60 seconds in the normal network conditions.
3	Simultaneous Transaction Safety	The system will not have race condition on simultaneous workflow actions on the same milestone. The isolation of transactions in PostgreSQL will be like in such a way that only s single state transaction is executed on an entity. Request packets that are violating the state order will be rejected and the relevant error messages will be sent.
4	API Responsiveness	The workflow API will be available during the I/O operation such as IPFS upload and blockchain confirmations. FastAPI patterns of asynchrony will provide that the waiting of external operation will not block other incoming requests. The API will be able to sustain 100 simultaneous request when the load is at peak.
5	Deliverable Integrity	The IPFS content addressing shall be used to provide deliverable integrity. The CID obtained on content of file shall be used as cryptographic evidence of submission. Any change on the file received will lead to a new CID and any attempt to tamper with it will be detected.
6	Security	Every endpoint to workflow will be configured to verify wallet signature prior to processing requests. Attempts of unauthorized access will be recorded with a timestamp, endpoint and actor wallet. The system will respond to unauthorized actions with error without disclosing sensitive information on the resource.

7	Error recovery	A periodic comparison of PostgreSQL state and smart contract state shall identify inconsistencies in this system. In cases where inconsistency is found the system will administratively endpoint resync database between contract state. All database changes should be avoided to ensure that in case of transaction failure the database remains unchanged.
8	Scalability	The workflow API will be scaled horizontally in stateless design. PostgreSQL will be able to provide querying of millions of milestone records with B-tree indexes that have sub 100ms response time. The IPFS pinning service will provide availability of deliverables on more than one node.
9	Availability	The workflow API will have 99% availability in the development and testing stages. The IPFS deliverables will have available via pinning services even when the application server is not available. Health check monitoring of rapid failure will be provided in critical workflow endpoints.

Table 3: Non-Functional Requirement

Chapter 4: System Diagram

4.1 Activity Diagram

This activity diagram represents the end-to-end view of the decentralized freelance platform. It starts by connecting a MetaMask wallet in order to authenticate the user. Once the user logs into the system, the activity splits into two separate processes; the client and freelancer. The client will provide a project then upload the related files to IPFS after that they deploy an escrow smart contract and finally they search for the relevant freelancer. At this point the freelancer will update their profile, browse a list of available projects once they see a list of projects they view the details of the project. This process joins once the client chose a freelancer and they both agree the contract then the client funds the smart contract. As soon as the freelancer send the deliverables to IPFS they add the CID on the smart contract. Then the client checks the work and if the work is satisfies, the payment is automated and freelancer reputation is updated and finally the project is completed otherwise if disputed the payment will not be unlocked until it has been solved. This activity diagram shows how the decentralized platform can organize storage, blockchain logic and user activity to give transparency.

Activity Diagram: Project Lifecycle (Search & Select Flow)

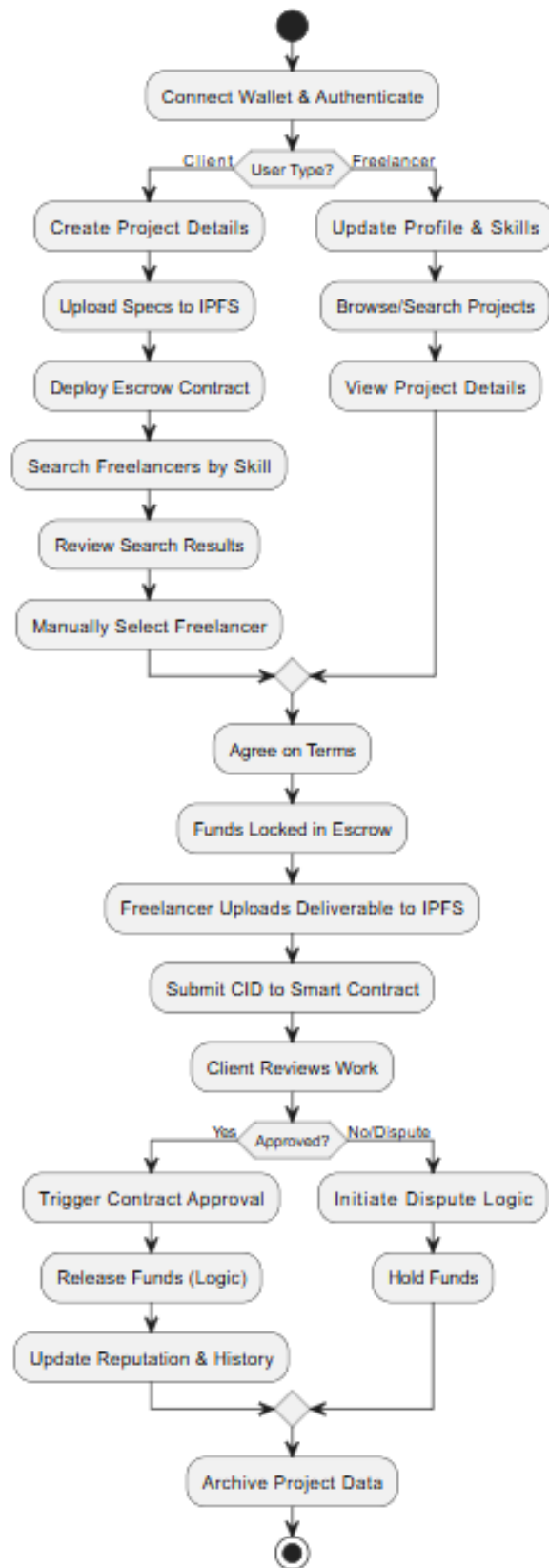


Figure 5: Activity Diagram for Decentralized Freelancing Platform

4.2 Sequence Diagram

The below sequence diagram explains the flow of decentralized freelancing platform into five major phases. It further display how all the parts of the system communicate during the whole process.

Firstly, the user connect his/her MetaMask wallet to the platform. After that frontend request digital signature of user wallet address for validating the user through backend cryptographic signature verification. Then only the user profile is created or updated in database and session authentication token along with user ID is provided. In second stage, client sends the project information from frontend which will then send request to the backend. Then the project file is uploaded to IPFS to get back the CID. The client will create a smart contract on the blockchain with project budget and rules used by CID and then the transaction hash will be recorded. Only after this project detail are stored in PostgreSQL with the CID.

In deliverable submission phase the freelancer submit their project work from the frontend. Then the file is uploaded to IPFS by backend to get the deliverables CID which is later sent to the smart contract with the project ID. After that smart contract record the deliverable CID and update project state. The client will review the work and then confirm the work only if its satisfies the requirement. After this process smart contract release funds to the freelancer and then release the completion event and return back to the frontend.

Finally in transparent verification phase user can verify the whole project history by querying the contract state from smart contract. The backend then fetch immutable log and metadata then display as the verified portfolio.

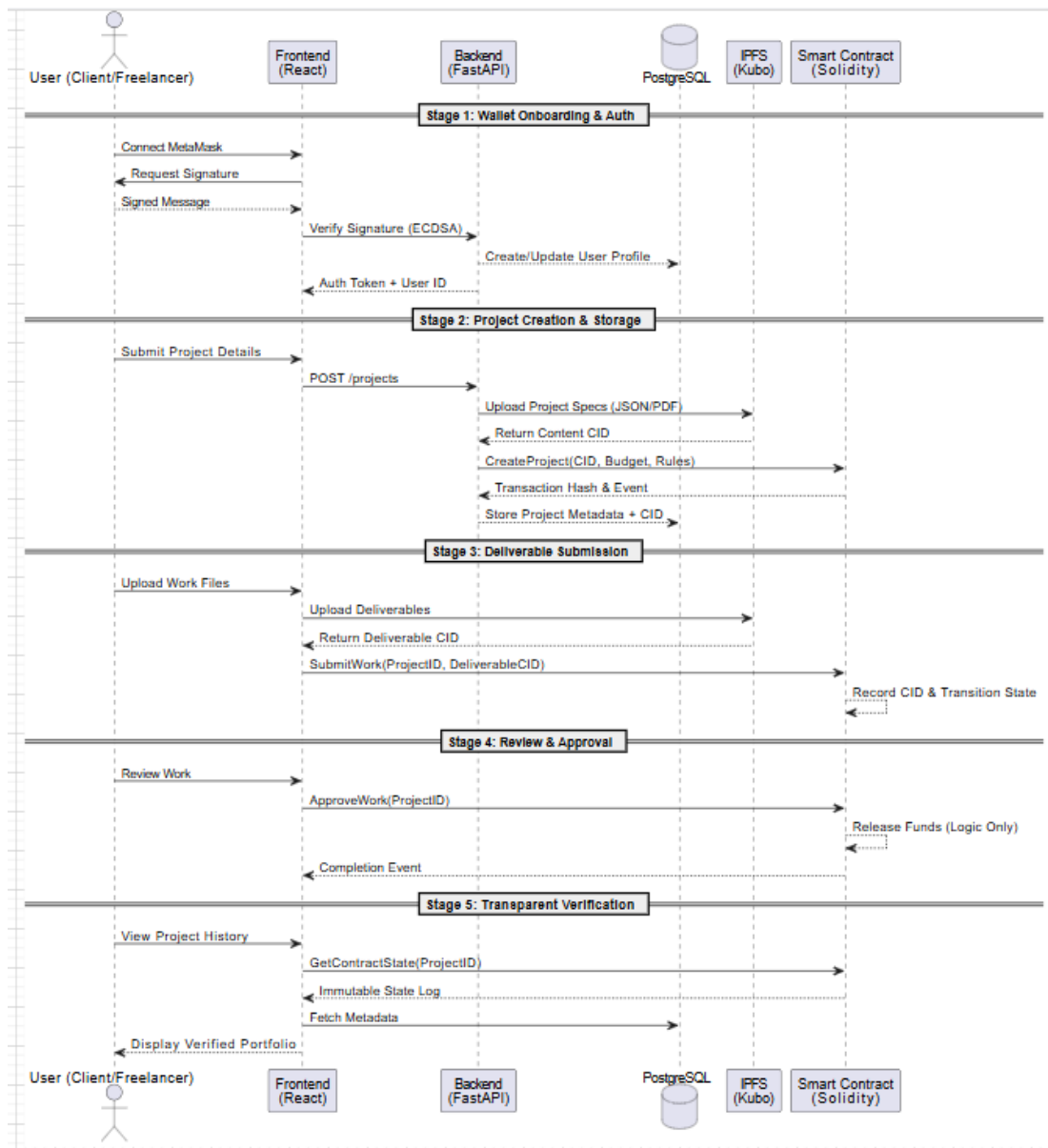


Figure 6: Sequence Diagram for Decentralized Freelancing Platform

4.3 Class Diagram

The above class diagram demonstrates the fundamental components and their relationship of the decentralized freelance platform. The system is divided into three categories such as user management, project management and technical infrastructure.

At the top of the diagram a user class define the foundation for two types of users i.e. Freelancer and Client and they share similar information like User ID and the Wallet Address. Even though they share similar information, each subclass has their own specific characteristics. For example: Freelancer data include skills, reputation, etc whereas Client may be able to send their project. The UserRole list all the possible functionalities for each type of User.

The Project class basically stores a key information like project title, status of and budget the project. A project is divided into 1 or more Milestones to track work progress and each Milestone has their own status and due date. The deliverables represent work handed by freelancer and then store the references to IPFS where all the files were stored. The ProjectStatus and MilestoneStatus identify all the possible state of entities like Created, Funded, In Progress, Submitted and Completed. The changes in all state are recorded through an AuditLog class.

The ApiController class on the right side acts an interface with all requests coming through frontend. This class assign most of the logic to the WorkflowCoordinator class because it controls the entire workflow. The StateValidator class examine whether all states are verified before transition is done and the AuthenticationService class authenticate all user operation based on their signature. The three adapter classes link the external services such as IPFSAdapter is responsible for interfacing with IPFS meanwhile BlockchainAdapter works with smart contracts. Based on ORM (Object Relational Mapping) the DatabaseRepository connects with the PostgreSQL database. The HybridDistributionAlgorithm helps to keep synchronization in all off-chain and on-chain operation because it helps in maintaining system state coherence.

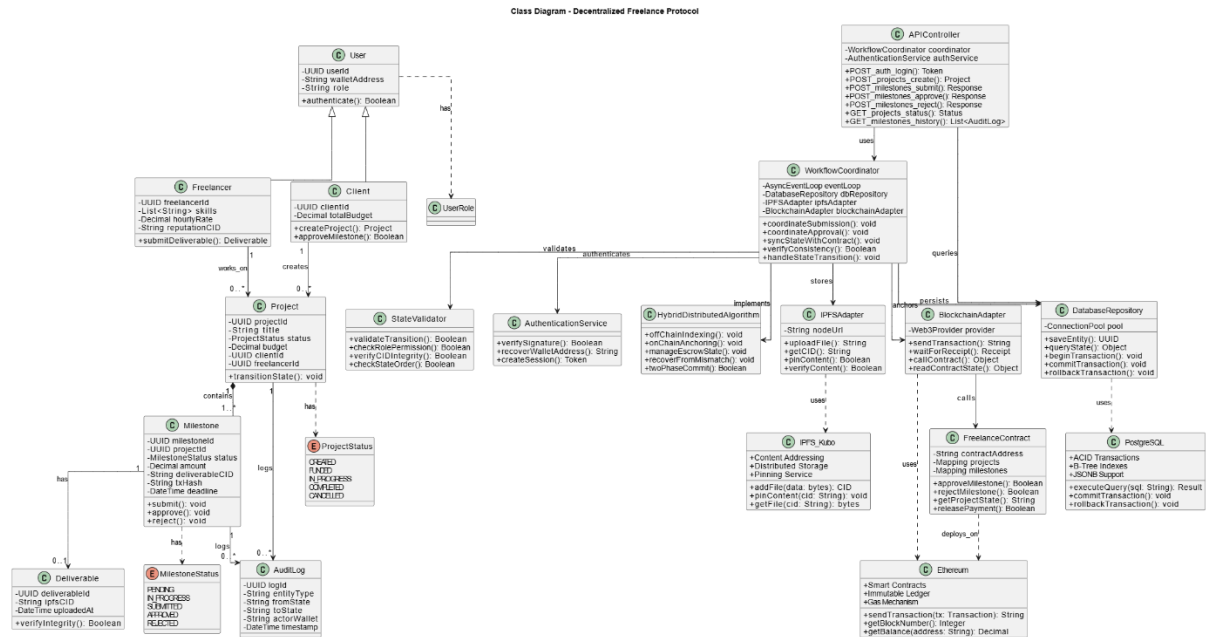


Figure 7: Class Diagram for Decentralized Freelance Platform

4.4 System Architecture Diagram

The system architecture diagram illustrates the workflow of decentralized freelance platform. A web application allows user to interact with system which is supported by the MetaMask wallet for managing blockchain interaction. When a user tries to create project and upload the work then the requests are sent to FastAPI backend which acts as the primary API endpoint that manages the system traffic for the system. This backend utilizes services of Hybrid Workflow Coordination because it is responsible for maintaining the state transitions from creation to funding and then to completion to in-progress of the projects and milestones. And it is also responsible for processing blockchain transactions and ensuring data is on sync across multiple data storage. The workflow uses the Two-Phase Commit Protocol that helps to ensures that all changes are updated.

The coordination layer communicates with 3 main storage mechanisms. The PostgreSQL database basically stores all metadata and the present state of the projects and milestone. And the IPFS system is use to stores all the files such as project information, deliverables and many more where each file is assigned a unique CID. The smart contracts that are stored in the Ethereum blockchain control important

elements such as managing the escrow accounts and releasing the funds depending on the work approval. The blockchain provide an immutable record for all of the transaction that happened on the platform. This hybrid model system interconnects the speed and security of the blockchain technology with the transparency of traditional databases.

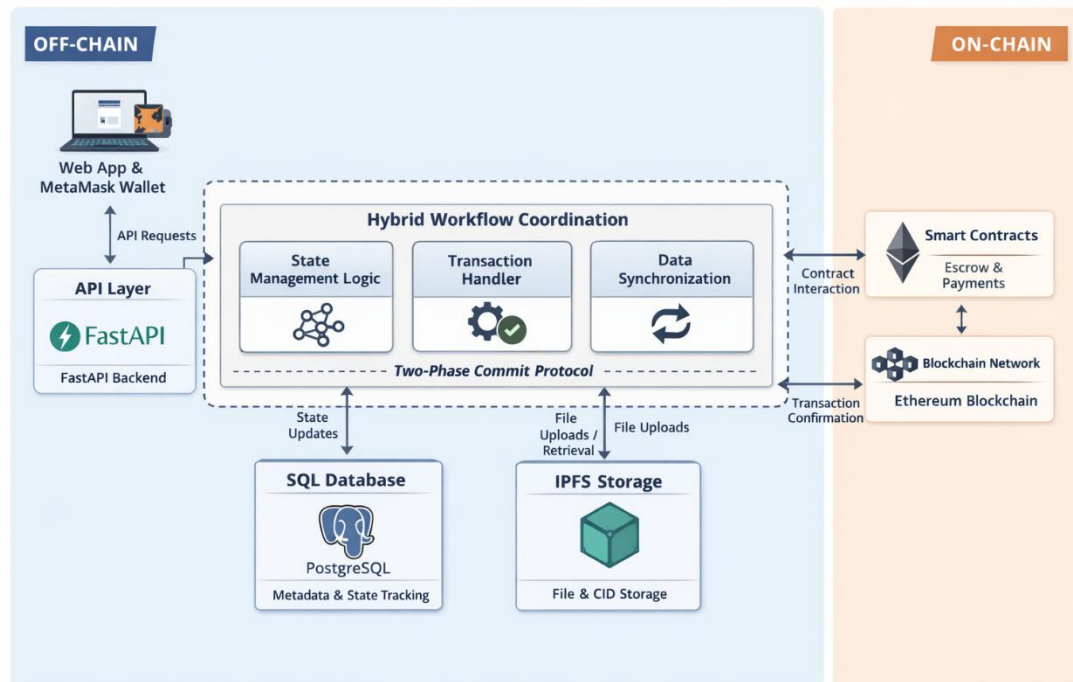


Figure 8: System Architecture Diagram

4.5 Use Case diagram

The use case diagram describes the various functionality which is accessible by different type of user in a decentralized freelancing platform. There are 3 primary actors within the system and they are Client, Freelancer and Admin.

A client has the many primary responsibilities. Firstly, they must authenticate with wallet in order to access the system functionality by validating client through cryptographical signature. The client can create a new project by defining the projects budget and milestones and then fund the projects with the desired budget. After that client review work submitted by freelancer and only approve each millstone that is satisfies to trigger smart contract approval. Finally, client verifies the deliverables CID for correct submission and rejecting each milestone if it is not satisfied with reason for

a rejection. The client can query transition history to see a full audit record of all states of each transaction.

A freelancer also has different responsibilities which include viewing projects to bid for them and when they are engaged on a project then freelancer can submit their deliverables to be revised by the client. Submitting deliverables includes the validation that the correct milestones were picked and then uploading only the necessary files to IPFS for decentralized storage. In addition, freelancers are also accessible to manage their profiles by updating their skills and viewing their reputation score as well as query the workflow state and transition history.

An admin is responsible for system monitoring which can be done by checking API status and database metrics. The admin is also responsible for managing users by viewing profiles and if necessary, they can suspend user account. They can solve issues by viewing dispute milestones and override transaction states between client and freelancers.

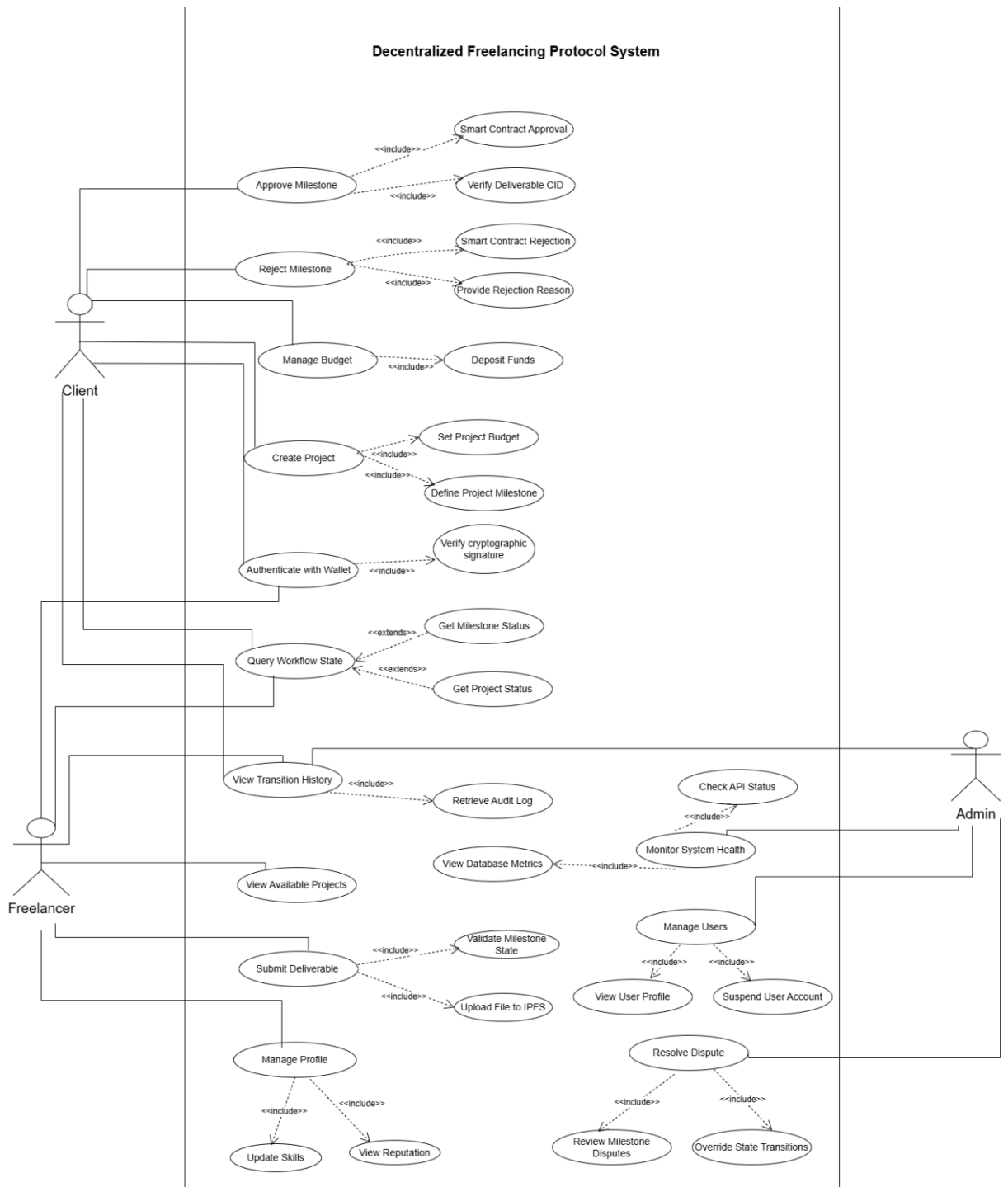


Figure 9: Use Case Diagram of Decentralized Freelancing Platform

4.6 Pseudocode

The following describes the logic of the main system functionalities. The pseudocode concentrates on the main workflow which involves wallet authentication, coordination of hybrid storage and state management over the blockchain.

1. Wallet Authentication and Login

```
ON connect Wallet Button Click
  GET Wallet Address from MetaMask
  Send Signature Request to User
  Receive Signed Message from User

  SEND POST Request to /api/auth/login
    BODY (wallet_address, signature)

  RECEIVE Response from Backend

  IF Response.status == success THEN
    STORE Authentication TOKEN in Local Storage
    REDIRECT USER to Dashboard
  ELSE
    SHOW "Signature Invalid"
  END
```

2. Creation of Project and Hybrid Storage

```
ON CREATE PROJECT Submit
  GET Project Details and Budget from Input
  SEND Project Files to IPFS
  RECEIVE Content Identifier (CID) from IPFS

  SEND POST Request to /api/projects/create
    BODY (details, budget, CID)

  IF Validation Success THEN
    Deploy Escrow Smart Contract with Budget
```

```

    RECEIVE Contract Address from Blockchain
    UPDATE PostgreSQL with Contract Address and Status "Funded"
    LOG Transaction in Audit Table
    SHOW Project Created Successfully
ELSE
    ROLLBACK Database Changes
    SHOW Project Creation Failed
END

```

3. Milestone Approval and Fund Release

```

ON Approve Milestone Click
    GET Milestone ID from Selection
    SEND POST Request to /api/milestone/approve

CHECK Current Status in PostgreSQL

IF Status == "Submitted" THEN
    Call Smart Contract Function "approveMilestone"
    WAIT for Transaction Confirmation

    IF Confirmation == Success THEN
        UPDATE PostgreSQL Status to "Approved"
        RELEASE Funds to Freelancer Wallet
        LOG Transaction in Audit Table
        SHOW Funds Released Successfully
    ELSE
        SHOW Blockchain Transaction Failed
    END
ELSE
    SHOW "State Invalid: Milestone is not Submitted"
END

```


References

- Bartłomiej Bednarz, M. M. (2025). Benchmarking the performance of Python web frameworks. Retrieved from <https://ph.pollub.pl/index.php/jcsi/article/view/7738/5148>
- Chong Guan, D. D. (2023). An ecosystem approach to Web3.0: a systematic review and research agenda. Retrieved from <https://www.emerald.com/jebde/article-pdf/2/1/139/1346506/jebde-10-2022-0039.pdf>
- Glin Tereshchenko, I. K. (2025). Hybrid System for Image Storage and Retrieval in Big Data Environments. Retrieved from <https://www.mecspress.org/ijigsp/ijigsp-v17-n3/IJIGSP-V17-N3-4.pdf>
- H. Luo, M. J. (2019). Construction Payment Automation through Smart Contract-based Blockchain Framework. Retrieved from https://www.researchgate.net/profile/Han-Luo-6/publication/334127210_Construction_Payment_Automation_through_Smart_Contract-based_Blockchain_Framework/links/5eba65de92851cd50dab6604/Construction-Payment-Automation-through-Smart-Contract-based-Blockchain-F
- Jadhav Sangram Bharat, P. S. (2025). Django vs. FastAPI: A Comparative Study for High-Performance Web Applications. Retrieved from <https://zenodo.org/records/15119179>
- Jinho Lim, K. N. (2021). A Study on Information Technology Freelancer Matching with Exploiting Blockchain in Gig Economy. Retrieved from https://dspace.hansung.ac.kr/bitstream/2024.oak/1870/2/AR_E1A1A4_A%20Study%20on%20Information%20Technology%20Freelancer%20Matching%20with%20Exploiting%20Blockchain%20in%20Gig%20Economy.pdf
- Kapoor, S. (2024). The Future of Web Development: Exploring JavaScript's Role in Web3 and Decentralized Apps. Retrieved from <https://ijsrm.net/index.php/ijsrm/article/view/5720/3543>
- Mihir Gandhi, P. S. (2019). Decentralized Freelancing System - Trust and Transparency. Retrieved from https://d1wqtxts1xzle7.cloudfront.net/60802666/IRJET-V6I934420191004-24611-14c1aeq-libre.pdf?1570258087=&response-content-disposition=inline%3B+filename%3DIRJET_Decimalized_Freelancing_System_T.pdf&Expires=1773981965&Signature=bl~NhmxsyuHBk7j27VktRY0fR
- Smita Athanere, R. T. (2022). Blockchain based hierarchical semi-decentralized approach using IPFS for secure and efficient data sharing. doi:<https://doi.org/10.1016/j.jksuci.2022.01.019>
- Tianqi Zhu, Y. H. (2026). Research on an On-Chain and Off-Chain Collaborative Storage Method Based on Blockchain and IPFS. doi:<https://doi.org/10.3390/fi18020092>

Appendix

Meeting Log 1:



Bachelors of Computer Science (Hons)
Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum **TEN (10)** for Capstone/FYP Part 1 and **TEN (10)** for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the **STUDENT** to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.

The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Runa Maphu Group: 19 Date: 22/02/2026
Meeting No: 1

Project Title: Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Subit Timalcina Supervisor's Signature: 

Item For Discussion: (Noted by Student Before Supervisory meeting)

- Overview of the project concept
- Clarification of individual roles and responsibilities

Record of Discussion: (Noted by Student During Supervisory meeting)

- Confirmed my individual role as the System Workflow Engineer.
- Addition of an API component to the project role

Action List (To be attempted or Completed by Student by the Next Supervisory Meeting)

- Research the specific responsibilities and workflow patterns
- Search for relevant academic research papers related to decentralized freelance platforms

***Note:** A student **MUST** attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under **NO** situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.*

Figure 10: Meeting Log 1

Meeting Log 2:



Bachelors of Computer Science (Hons)
Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum TEN (10) for Capstone/FYP Part 1 and TEN (10) for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session. The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Runa Maphu	Group: 19	Date:
Meeting No: 2		

Project Title: Decentralized Freelance Protocol with Web3 Integration	
Supervisor's Name: Subit Timalina	Supervisor's Signature: 

Item For Discussion: (Noted by Student Before Supervisory meeting)

- Presenting the research articles found regarding decentralized platforms. Checking if the articles are technically relevant to the project scope.

Record of Discussion: (Noted by Student During Supervisory meeting)

- The supervisor reviewed the research articles and confirmed they are relevant.
- The supervisor discussed format for the review paper, explaining how to structure the analysis of the articles

Action List (To be attempted or Completed by Student by the Next Supervisory Meeting)

- Prepare 2-3 drafts of the review paper.
- Bring the drafts to the next meeting for a detailed review

***Note:** A student **MUST** attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under NO situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.*

Figure 11: Meeting Log 2

Meeting Log 3:



Bachelors of Computer Science (Hons)
Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum **TEN (10)** for Capstone/FYP Part 1 and **TEN (10)** for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.

The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Runa Maphu Group: 19 Date:

Meeting No: 3

Project Title: Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Subit Timalina Supervisor's Signature: 

Item For Discussion: (Noted by Student **Before** Supervisory meeting)

- Presenting the draft of the review paper.
- Reviewing the synthesis of the collected research articles.

Record of Discussion: (Noted by Student **During** Supervisory meeting)

- The supervisor reviewed the draft of the review paper.
- Advised me to rearrange the format of the review paper draft.

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)

- Bring the newly formatted draft to the next meeting.

***Note:** A student **MUST** attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under NO situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.*

Figure 12: Meeting Log 3

Bachelors of Computer Science (Hons)
Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum **TEN (10)** for Capstone/FYP Part 1 and **TEN (10)** for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.

The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Runa Maphu	Group: 19	Date:
Meeting No: 4		

Project Title: Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Subit Timalsina

Supervisor's Signature:

Item For Discussion: (Noted by Student **Before** Supervisory meeting)

- Presenting the final literature review paper.
- Discussing the structure of the literature review table and the system diagram.
- Planning the SWOT analysis for the project.

Record of Discussion: (Noted by Student **During** Supervisory meeting)

- The final version of the literature review was reviewed and accepted.
- Discussed the format for the summary review table
- Reviewed the strengths, weaknesses, opportunities, and threats related to the project's technical implementation.

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)

- Complete the project report up to the SWOT analysis section.

Note: A student **MUST** attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under NO situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.

40 | Page

Meeting Log 5:



Bachelors of Computer Science (Hons)
Capstone Project Log Sheet

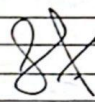
Notes on use of the project log sheet:

1. Minimum **TEN (10)** for Capstone/FYP Part 1 and **TEN (10)** for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the **STUDENT** to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session. The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Runa Maphu Group: 19 Date:

Meeting No: 5

Project Title: Decentralized Freelance Protocol with Web3 Integration

Supervisor's Name: Subit Timalina Supervisor's Signature: 

Item For Discussion: (Noted by Student **Before** Supervisory meeting)

- Reviewing the SWOT analysis draft.
- In-depth discussion on technical frameworks (FastAPI and Web3.py)
- Planning the system's structural design.

Record of Discussion: (Noted by Student **During** Supervisory meeting)

- Discussed adding a tool comparison to the SWOT analysis to justify the technology choices.
- Evaluated which features of FastAPI and Web3.py are most relevant.
- Discussed the UML diagrams to ensure the system architecture is correctly mapped.

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)

- Update the draft of the SWOT analysis with the tool comparison
- Refine the UML diagrams based on the technical discussion.

***Note:** A student **MUST** attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under **NO** situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.*

Figure 14: Meeting Log 5

Meeting Log 6:



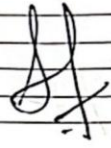
Bachelors of Computer Science (Hons)
Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum TEN (10) for Capstone/FYP Part 1 and TEN (10) for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.

The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Runa Maplu	Group: 19	Date:
Meeting No: 6		

Project Title: Decentralized Freelance Protocol with Web3 Integration	
Supervisor's Name: Subit Timalina	Supervisor's Signature: 

Item For Discussion: (Noted by Student Before Supervisory meeting)

- Presentation of the technical tools selected (FastAPI, Web3.py, PostgreSQL).
- Review of the SWOT analysis draft for the decentralized platform.
- Discussion on the architectural requirements for the system.

Record of Discussion: (Noted by Student During Supervisory meeting)

- Discussed which technical tools are most relevant to the project's specific need.
- Reviewed the SWOT analysis and identified areas to strengthen in the report.
- Discussed the initial logic for the UML diagrams to ensure the system workflow is accurate.

Action List (To be attempted or Completed by Student by the Next Supervisory Meeting)

- Complete the project report draft up to the SWOT analysis section.
- Create the UML diagrams (Use Case and Class diagrams) based on the discussion.

*Note: A student **MUST** attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under NO situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.*

Figure 15: Meeting Log 6

Meeting Log 7:



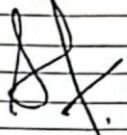
Bachelors of Computer Science (Hons)
Capstone Project Log Sheet

Notes on use of the project log sheet:

1. Minimum **TEN (10)** for Capstone/FYP Part 1 and **TEN (10)** for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the **STUDENT** to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session. The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student **must** hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Runa Mapthu Group: 19 Date:

Meeting No: 7

Project Title: Decentralized Freelance Protocol with Web3 Integration	
Supervisor's Name: Subit Timalisina	Supervisor's Signature: 

Item For Discussion: (Noted by Student **Before** Supervisory meeting)

- Review of the system architecture and user interaction flow.

Record of Discussion: (Noted by Student **During** Supervisory meeting)

- The initial UML diagrams were reviewed to ensure all system actors (Freelancers, Clients) are correctly represented.
- Received feedback on specific areas of the diagram that need to be more detailed for the final report.

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)

- Update and fix the UML diagrams based on the discussion.
- Bring the updated UML documentation to the next meeting.

***Note:** A student **MUST** attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under **NO** situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.*

Figure 16: Meeting Log 7

Meeting Log 8:



Bachelors of Computer Science (Hons)
Capstone Project Log Sheet

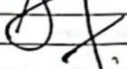
Notes on use of the project log sheet:

1. Minimum **TEN (10)** for Capstone/FYP Part 1 and **TEN (10)** for Capstone 2
2. The student should prepare for the supervisory sessions by deciding which question(s) he or she needs to ask the supervisor and what progress has been made (if any) since the last session and note these in the relevant sections of the form, effectively forming an agenda for the session.
3. A log sheet is to be brought by the STUDENT to each supervisory session.
4. The actions by the student (and, perhaps the supervisor), which should be carried out before the next session should be noted briefly in the relevant sections of the form.
5. The student should make copies of the Log Sheet and must be inserted in the documentation (under appendices) in the softbound report.
6. It is recommended that students bring along log sheets of previous meetings during each supervisory session.

The log sheet is an important delivery for the research and an important record of a student's organization and learning experience. The student must hand in the log sheets as an appendix of the documentation, with sheets dated and numbered consecutively.

Student Name: Runa Maphu	Group: 19	Date:
Meeting No: 8		

Project Title: Decentralized Freelance Protocol with Web3 Integration	
---	---

Supervisor's Name: Subit Timalisina	Supervisor's Signature: 
-------------------------------------	---

Item For Discussion: (Noted by Student **Before** Supervisory meeting)

- Presenting the finalized version of the project report.
- Ensuring all sections (Literature Review, SWOT, UML, etc.) are complete

Record of Discussion: (Noted by Student **During** Supervisory meeting)

- The entire project report was reviewed in detail.
- Received confirmation that the report meets the required standards

Action List (To be attempted or Completed by Student by the **Next** Supervisory Meeting)

- Prepare the final file format for official submission.

***Note:** A student **MUST** attend all weekly meetings as scheduled by the supervisor. In the event Supervisor could not be met, the student must immediately inform the Academic Coordinator and supervisor so that a meeting can be subsequently arranged. Under NO situation should a student miss a weekly appointment with the supervisor except during exceptional circumstances.*

Figure 17: Meeting Log 8

Turnitin Similarity Report

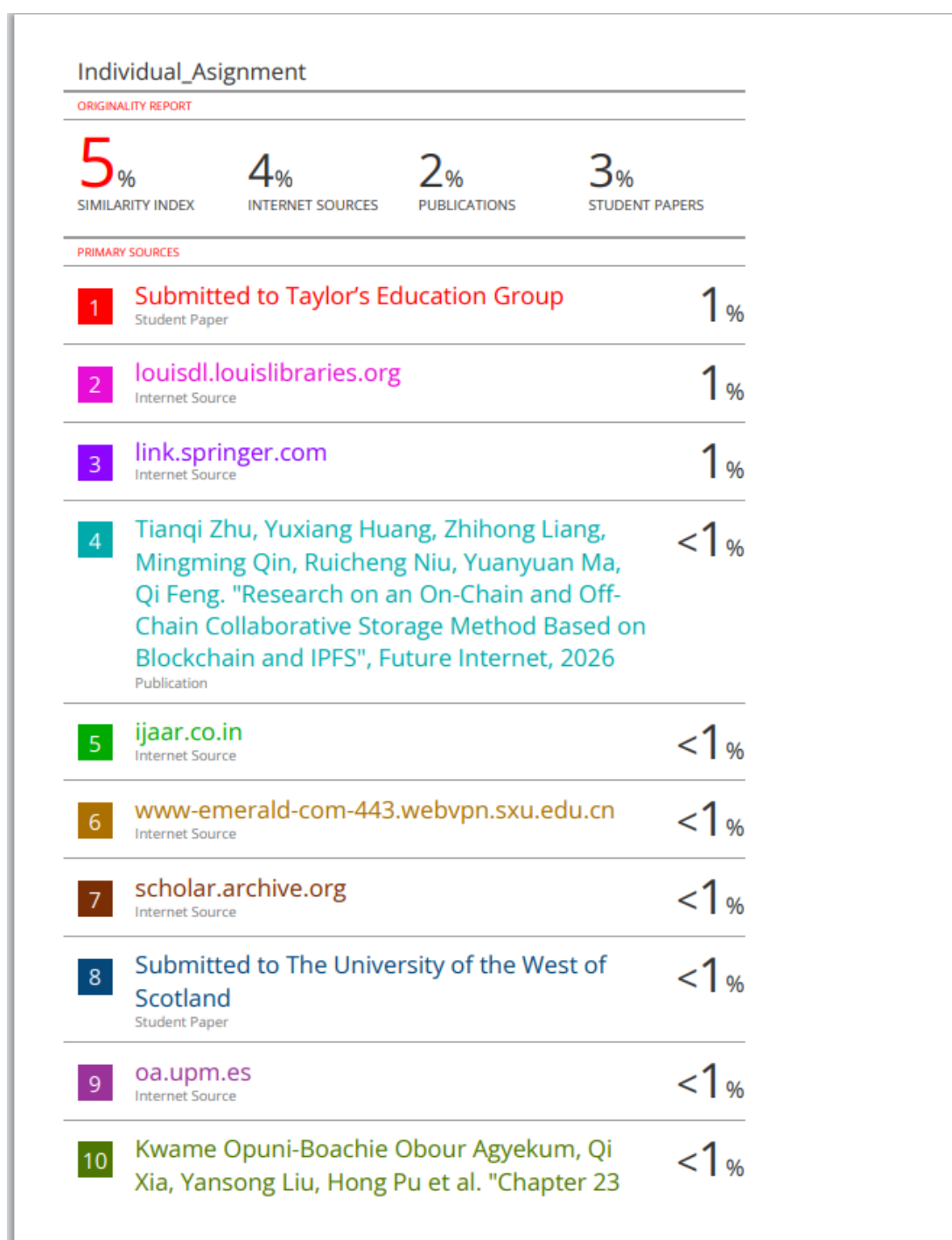


Figure 18: Turnitin Similarity Report

Digital Media Copyright and Content
Protection Using IPFS and Blockchain",
Springer Science and Business Media LLC,
2019
Publication

11	Submitted to University of New South Wales Student Paper	<1 %
12	www.ijstrm.net Internet Source	<1 %
13	Submitted to Berlin School of Business and Innovation Student Paper	<1 %
14	Submitted to University of Johannesburg Student Paper	<1 %
15	dspace.nm-aist.ac.tz Internet Source	<1 %
16	su-plus.strathmore.edu Internet Source	<1 %
17	www.mecs-press.org Internet Source	<1 %
18	Submitted to Coventry University Student Paper	<1 %
19	purehost.bath.ac.uk Internet Source	<1 %

Exclude quotes Off

Exclude matches Off

Exclude bibliography Off

Turnitin Digital Receipt



Digital Receipt

This receipt acknowledges that Turnitin received your paper. Below you will find the receipt information regarding your submission.

The first page of your submissions is displayed below.

Submission author: RUNA MAPHU .
Assignment title: Assessment Task 2: Individual Capstone Project Report (40%) ...
Submission title: Individual_Assignment
File name: 243065_RUNA_MAPHU_-_Individual_Assignment_867485_79741...
File size: 1.29M
Page count: 30
Word count: 8,255
Character count: 44,846
Submission date: 01-Apr-2026 11:17AM (UTC+0800)
Submission ID: 2919423157



Copyright 2026 Turnitin. All rights reserved.

Figure 19: Turnitin Digital Receipt

Proof of Diagrams:

1. SWOT Table

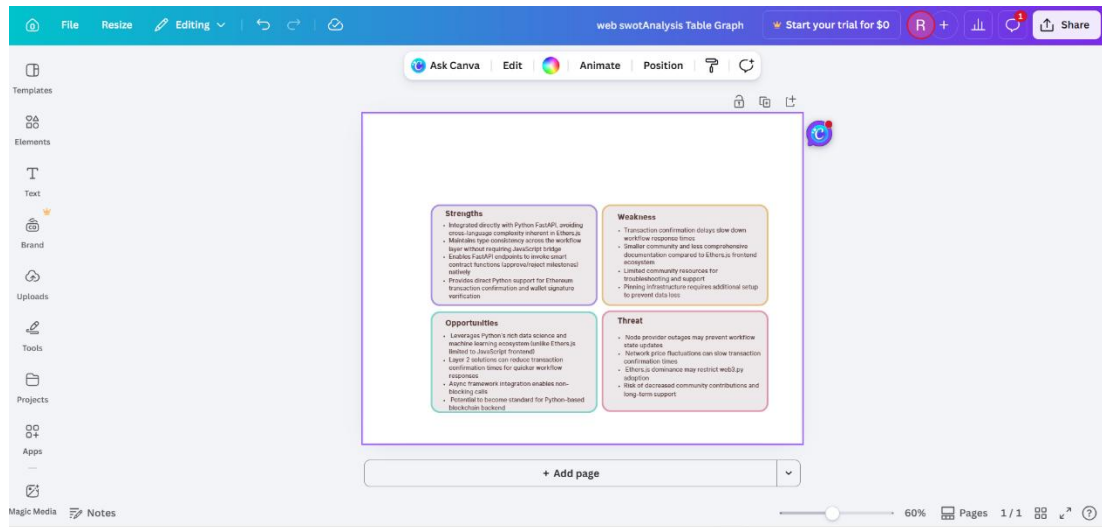


Figure 20: Proof of SWOT Table

2. System Architecture

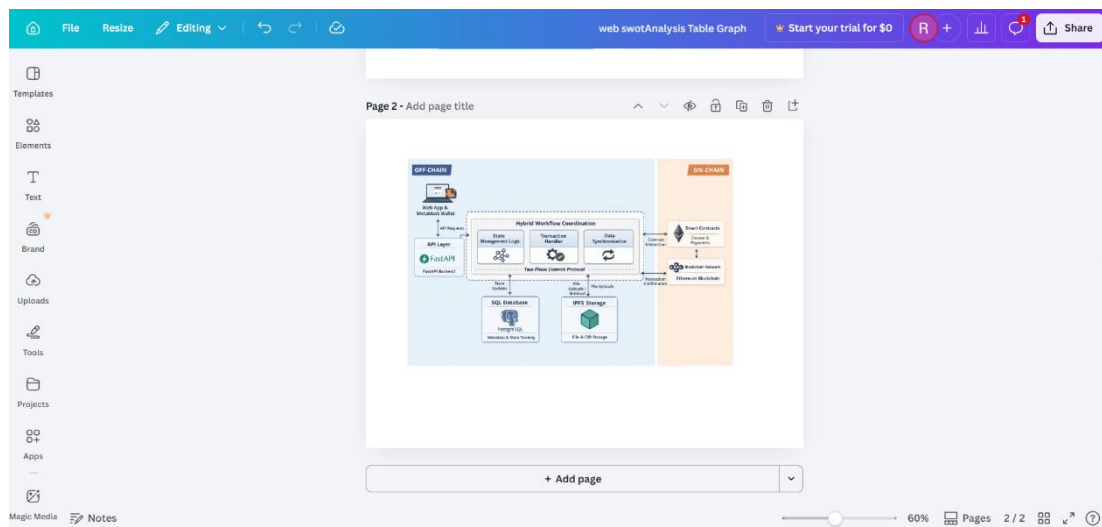


Figure 21: Proof of System Architecture Diagram

5. Class Diagram

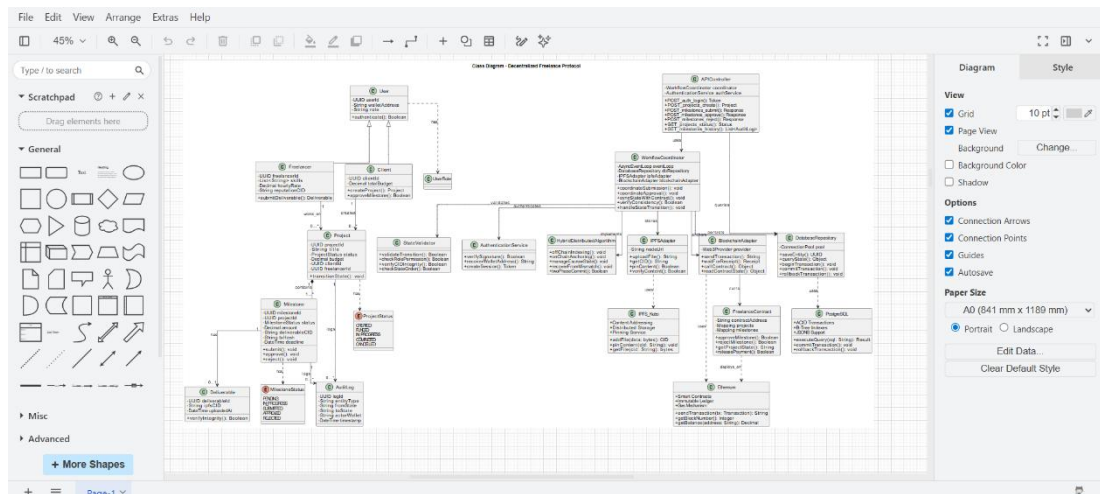


Figure 24: Proof of Class Diagram

6. Use Case Diagram

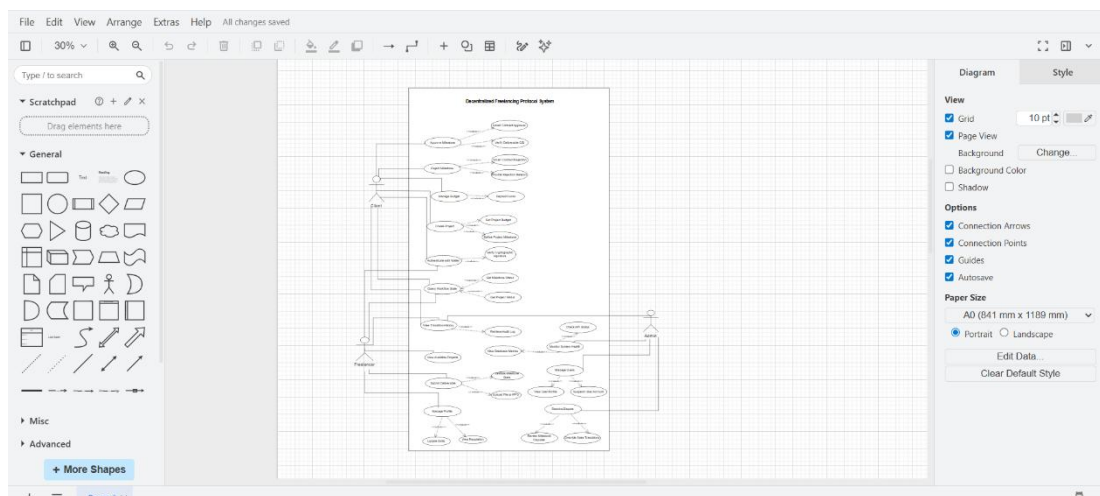


Figure 25: Proof of Use Case Diagram